

# Computing for Analytical Work

**Session 3 — What broke?**

**Block 6 — Data files, formats, size, and recovery**

## **This block in one sentence**

**Data problems often look like code problems. Inspect the input before blaming the code.**

The code is the same today as yesterday. The file is not. Look at the file.

## What we cover

1. Data files as external reality
2. What a CSV really is: delimiters and header rows, and one prediction
3. Look at the file *before* you load it
4. Five ways a CSV lies, and the `read_csv` option for each — including the one that lies quietly
5. The four-line sanity check
6. Git thread — the final clean submission
7. AI thread — hypotheses from the model, validation from the data
8. Lab 6 — The data is the bug, then Homework 3

## Data files as external reality

- Your code encodes **assumptions**: this column exists, this is a number, this is a date
- The file was made by someone else, on another machine, with another tool, possibly by hand
- The file does not know about your assumptions and does not care
- Same code, new file, new failure — or worse, new *silent* wrong answer
- Session 1 taught you the filesystem is the source of truth. So is the file's **content**

# What a CSV actually is

```
$ head -3 customers.csv
```

```
id,name,city ②  
1,Anna,Vienna ①  
2,Bela,Budapest ③
```

pd.read\_csv()

what pandas built

| id | name | city     |
|----|------|----------|
| 1  | Anna | Vienna   |
| 2  | Bela | Budapest |

df.shape → (2, 3)

## ① the delimiter

a comma here — by convention, not by law

## ② the header row

the first line; a header only because read\_csv assumes so

## ③ one row per line

each line splits at the delimiter into the columns

`pd.read_csv` assumes: comma, header on line 1, a handful of spellings for *missing*. Every assumption is a place the file can disagree.

## Look at the file before you load it

```
head -5 data/raw/customers.csv      # first five lines: delimiter, header, first values
tail -3 data/raw/customers.csv      # last three: a footer, a totals row?
wc -l data/raw/customers.csv        # how many lines
file data/raw/customers.csv         # what kind of file it really is
```

- **Before you load, in the terminal:** these four, thirty seconds, before a line of Python. All work in Git Bash
- **After you load, in Python:** `df.shape` against `wc -l` — the header and any title lines are lines too
- They read only what you ask for, so they work on a file **too big for Excel** to open
- Ask the file questions; do not scroll through it. Opening the whole file is not inspecting it

## Predict: how many columns?

```
id;name;city  
1;Anna;Vienna  
2;Bela;Budapest
```

```
df = pd.read_csv("data/raw/customers.csv")  
print(df.shape)
```

That is `head -3` of the file, and the load with no options. What does `df.shape` print — and is there an error? Write it down. Do not type.

## Reveal: wrong delimiter, and no error at all

```
$ head -3 customers.csv
```

```
id;name;city  
1;Anna;Vienna  
2;Bela;Budapest
```

read\_csv()

what pandas built

| id;name;city    |
|-----------------|
| 1;Anna;Vienna   |
| 2;Bela;Budapest |

df.shape → (2, 1)

df.columns → ['id;name;city']

no error, no warning

**The signature: one column where you expected several.**

Learn to read it from the shape alone.

### 1 No commas found

so every line is one field: one column, whose name is the whole header line. The next line of your code, `df["name"]`, raises `KeyError` — and you go looking for a bug in the code.

## The fix is one argument

```
df = pd.read_csv("data/raw/customers.csv", sep=";")  
df = pd.read_csv("data/raw/export.tsv", sep="\t")  
print(df.shape)           # (2, 3)
```

- `sep=` tells pandas what the delimiter really is
- You knew which one to pass because you ran `head -5` first
- One column where you expected many is the signature — learn to recognise it

## Extra header rows

```
$ head -4 orders.csv
Orders export
Generated 2026-03-01
order_id,customer_id,amount,order_date
1001,7,"€1,200",2026-01-05
```

### 1 Two lines that are not data

sit above the real header. Read the error's numbers against the file: expected 1, saw 4, line 3 — the numbers are the map.

read\_csv()

what pandas counted

line 1 → 1 field becomes the header: one column, 'Orders export'  
 line 2 → 1 field a row: fine, one column  
 line 3 → 4 fields does not fit one column: stop

**ParserError: Error tokenizing data. C error:**  
**Expected 1 fields in line 3, saw 4**

**skiprows=2 tells pandas where the header really is.**

header=None, names=[...] is the opposite case: a file with no header at all.

```
df = pd.read_csv("data/raw/orders.csv", skiprows=2) # junk above the header
df = pd.read_csv("data/raw/bare.csv", header=None, names=["id", "name", "city"]) # no header at all
```

## Missing values, and the many spellings of missing

```
df = pd.read_csv("data/raw/orders.csv", na_values=["-", "?", "missing"])
print(df.isna().sum())
```

- pandas recognises some spellings by default: an empty cell, `NA`, `N/A`, `n/a`, `NaN`, `null`, `None`
- It does **not** recognise `-`, `?`, `missing`, `.`, or `999` — those stay as text
- One `-` in a numeric column turns the **whole column** into text
- `na_values=` declares your file's spellings; then `isna().sum()` — a **count per column**, not the rows themselves — shows how many you just declared missing, a number you should be able to justify

# Numbers stored as text

```
$ head -4 orders.csv
order_id,amount
1001,"€1,200" ①
1002,- ②
1003,830.50
```

## ① A currency sign and a thousands comma

make 1200 a string. pandas loads it faithfully as text and says so in dtypes.

## ② One dash for "missing"

is not a spelling pandas knows; it stays text — and one text value makes the whole column text, 830.50 included.

pd.read\_csv()

what pandas built

| order_id | amount |
|----------|--------|
| 1001     | €1,200 |
| 1002     | -      |
| 1003     | 830.50 |

df.dtypes → order\_id    int64

amount    str

(an older pandas prints object)

**The tell: a column that should be numeric says str.**

Strip the symbols, declare the missing spelling (na\_values), then convert.

```
df["amount"] = pd.to_numeric(df["amount"].str.replace("[€,]", "", regex=True), errors="coerce")
print(df["amount"].isna().sum())    # how many became NaN — and can you name every one of them?
```

## Dates as strings — name the formats you saw

```
iso = pd.to_datetime(df["order_date"], format="%Y-%m-%d", errors="coerce")
dmy = pd.to_datetime(df["order_date"], format="%d/%m/%Y", errors="coerce")
df["order_date"] = iso.fillna(dmy)
print(df["order_date"].isna().sum()) # how many matched neither format? Zero, or you missed one
```

- A date column with dtype `str` is text, and sorting text dates gives nonsense — as does sorting numbers stored as text: `"100"` before `"20"`
- `format=` names one exact layout; `errors="coerce"` turns what does not match into `NaT` instead of stopping
- Two layouts in one column, which `head` showed you, means two calls, combined — then the count
- `05/01/2026` is ambiguous on its own. **Find one slash date whose first number is above 12:**  
`24/05/2026` can only be day-first, and one author wrote them all

## `format="mixed"` — the guess that goes wrong quietly

```
pd.to_datetime(["2024-01-06", "04/02/2024"], format="mixed", dayfirst=True)
# → 2024-06-01, 2024-02-04          ← the ISO date was flipped: June, not January
pd.to_datetime(["2024-01-06", "04/02/2024"], format="mixed")
# → 2024-01-06, 2024-04-02          ← the slash date was flipped: April, not February
```

- The error message you get on a mixed column **suggests** `format="mixed"`. It guesses a layout per value
- For any date whose day is 12 or less, the guess can go either way — **no error, no warning**, wrong month
- `dayfirst=True` fixes the slash dates and breaks the ISO ones. Neither setting is right for this file
- You looked at the file. You know the layouts. **Name them**. Guessing is what you do when you have not looked

## The four-line sanity check

```
df = pd.read_csv("data/raw/sales.csv")
print(df.shape)      # (rows, columns) – is this the size you expected?
print(df.dtypes)      # numbers, not text? dates, not text?
print(df.head())      # what does the data actually look like?
print(df.isna().sum()) # which columns have gaps, and how many?
```

**Run it on every dataframe, every time — as reflexively as you hit save.**

## What each line catches

- `df.shape` → wrong delimiter (1 column), skipped rows, a truncated file
- `df.dtypes` → numbers as text, dates as text, one stray `–` poisoning a column
- `df.head()` → junk header rows, misaligned fields, a currency symbol you did not expect
- `df.isna().sum()` → unrecognised spellings of missing, coercion that silently ate values
- Also look at: `df.columns.tolist()` for names, `df["col"].unique()[:10]` — the first ten distinct values — for a stray `–` or `€`, `df["date"].min()` and `.max()` for a guess that went wrong
- Compare every number to what you expected **before** you loaded the file

## Symptom in the code, cause in the file

| You see                                                        | Look at the file for                                                   |
|----------------------------------------------------------------|------------------------------------------------------------------------|
| <code>KeyError: 'name'</code> after load                       | wrong delimiter, extra header lines, different header spelling         |
| <code>ValueError: could not convert</code>                     | currency symbols, thousands separators, a <code>-</code> for missing   |
| <code>TypeError</code> on <code>+</code> or <code>sum()</code> | a text column that should be numeric                                   |
| A total that is 10× too small                                  | rows dropped by <code>skiprows</code> , or coerced to <code>NaN</code> |
| A month with too many orders, and no error                     | a date layout that was guessed instead of named                        |

## Not today: Excel, JSON, encodings, file size

- All four are real and you will meet them this year
- They live on the **Lab 6 reference sheet** — `REFERENCE-other-formats.md` in the lab repo
- Excel: why opening data in Excel silently changes it
- JSON: what malformed means and how to spot it
- `encoding="latin-1"` when `utf-8` fails on the first byte
- Size: why a file may not open in Excel at all, and what to do instead

## **Git thread — the final clean submission**

## git status → git diff → commit → git status

```
git status
git diff
git add .
git commit -m "Load customers with sep=';"
git status
```

- **status** : what is modified, what is new, what is junk
- **diff** : read **every** changed line — is each one something you meant?
- This diff has two green lines. One is the fix. The other is debris

```
anna@laptop project $ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)

        modified:   scripts/load.py

no changes added to commit (use "git add" and/or "git commit -a")
anna@laptop project $ git diff
diff --git a/scripts/load.py b/scripts/load.py
index 8355d8a..3eb1a16 100644
--- a/scripts/load.py
+++ b/scripts/load.py
@@ -1,5 +1,6 @@
import pandas as pd

-df = pd.read_csv("data/raw/customers.csv")
+df = pd.read_csv("data/raw/customers.csv", sep=";")
+print(df.columns)
assert df.shape[1] == 3, df.columns
print(df.groupby("region").size())
anna@laptop project $
```

## Before submitting, know exactly what changed

- The debris is gone; the diff is one line, the one you meant
- A message that says **what** and **why**, then `status` once more: *nothing to commit, working tree clean*
- Generated output and `.venv/` should be ignored, not committed; `status` shows if they are not
- `git log --oneline` should read like the story of the repair
- Homework 3 asks for this: a clean tree, and a log with one commit per fix

```
anna@laptop project $ git diff
diff --git a/scripts/load.py b/scripts/load.py
index 8355d8a..d13de3b 100644
--- a/scripts/load.py
+++ b/scripts/load.py
@@ -1,5 +1,5 @@
     import pandas as pd

    -df = pd.read_csv("data/raw/customers.csv")
    +df = pd.read_csv("data/raw/customers.csv", sep=";")
    assert df.shape[1] == 3, df.columns
    print(df.groupby("region").size())
anna@laptop project $ git add .
anna@laptop project $ git commit -m "Load customers with sep=; - the
export is semicolon-delimited"
[main 8bd0084] Load customers with sep=; - the export is semicolon-d
elimited
    1 file changed, 1 insertion(+), 1 deletion(-)
anna@laptop project $ git status
On branch main
nothing to commit, working tree clean
anna@laptop project $ █
```

**AI thread — hypotheses from the model, validation from the data**

## Three good questions to ask

1. Here is the traceback and the first five lines of the file (head -5).  
What are the likely causes of this parsing error?
2. I have a 2 GB CSV I have never seen. How do I inspect it safely without opening it in Excel or loading all of it?
3. What does `skiprows=` do in `pd.read_csv`, and when would I use `header=` instead?

Each one comes back with hypotheses or explanations — **not** with a verified dataframe.

## Validation comes from inspecting the data

- The model suggests `sep=";"`. Did `df.shape` change from `(500, 1)` to `(500, 6)`?
- The model suggests `na_values=["-"]`. Does `isna().sum()` show a count you can explain?
- The model suggests `errors="coerce"`. How many values became `NaN`, and which ones?
- The model suggests a date parser. Do `min()` and `max()` span what the file spans, and did nothing become `NaT`?
- **Verification standard:** row counts, column names, dtypes, sample records, the output file exists
- Your AI-use note says what you asked and which of these you checked

## **5-minute stand-and-stretch**

Stand up. Leave the laptop. Back in five for Lab 6.

## Lab 6 — The data is the bug

In your terminal (Git Bash on Windows, Terminal on macOS), in the folder where you keep course work:

```
git clone https://github.com/earino/ecbs5293-lab06-data-files.git
cd ecbs5293-lab06-data-files
head -5 data/raw/customers.csv
head -5 data/raw/orders.csv
wc -l data/raw/*.csv
uv sync
uv run python scripts/load.py
```

`scripts/load.py` is mostly reasonable. The files are not: a delimiter, two title lines, amounts like "€1,200", dates in two layouts, missing spelled two ways. Fix each with a **loading option**, never by editing the file.

```
Traceback (most recent call last):
  File "/Users/Shared/anna/ecbs5293-lab06-data-files/scripts/load.py", line 45, in <module>
    main()
    ~~~~^
  File "/Users/Shared/anna/ecbs5293-lab06-data-files/scripts/load.py", line 33, in main
    customers = load_customers()
  File "/Users/Shared/anna/ecbs5293-lab06-data-files/scripts/load.py", line 19, in load_customers
    assert df.shape[1] == 4, f"expected 4 customer columns, got {df.shape[1]}: {list(df.columns)}"
           ^^^^^^^^^^^^^^^
AssertionError: expected 4 customer columns, got 1: ['customer_id;name;region;signup_date']
anna@laptop ecbs5293-lab06-data-files $
```

## Run it, fix the load, validate, note it

1. Load each file with the **right options**: `sep=`, `skiprows=`, `na_values=`, strip-then-convert, two named date formats
2. **Validate**: row count against `wc -l` minus the non-data lines; column names; dtypes; `isna().sum()` explainable; date `min()` / `max()` plausible
3. `output/revenue_by_region.csv` exists; **one note per data problem** in `DIAGNOSIS.md`, laid out for you — paste each message **before** you change the load
4. `NEXT_TIME.md`: three things that could break next time, and the check that catches each
5. `git status`, `git diff` — read it — commit, `git status` clean
6. **Minute 33**: explain one file's problems and your validation to a neighbour. Checkpoint on Moodle before you leave

## The last ten minutes — explain it to a neighbour

**Speaker, about a minute, pointing at the screen, not reading the note:** symptom → cause → evidence → change → verification.

**Listener asks:**

1. Show me the evidence.
2. Why did it fail — not just where?
3. Next month's export gains a fifth column at the end, and nothing else changes. Which of your four checks fires, and does your loading fix still run?

Swap. Unsure, or you disagree? **Hand up.** Then the answer, for everyone. Before you leave: **DIAGNOSIS.md** to the Lab 6 checkpoint on Moodle.

## Homework 3 assigned

### Debugging, data files, recovery, and final diagnosis note

- Budget about **7–8 hours**, over the two weeks
- Due: **Fri 9 Oct, 23:59** — not this Friday, the one after; one day late at –10%; the self-check key posts Sunday 11 Oct
- A repo with several realistic failures: **some are code, some are data**
- Repair it, verify the output, explain every failure — and tell code problems from data problems
- Then a **mock exam** in the real format, with a key. The exam is in **reading week**.

## Homework 3 — deliverables

1. Fixed repo with a **clean end-to-end run** from a fresh unzip
2. The **four-line check** after each load, left in the code, and `output/report.csv` sensible — look at its months
3. One **diagnosis note per failure**, each labelled **code** or **data**, with the evidence that decided it — pasted before you fixed it
4. `AI_USE.md` — what you asked, how you verified, if you used it
5. `REFLECTION.md` — the end-of-course reflection, ~300 words, next slide
6. **One commit per fix**, a clean tree, and the archive by `SUBMITTING.md`, which ends with a seven-line check
7. **Video walkthrough, up to 2 minutes** — run it end to end, then the failures, telling data from code

## End-of-course reflection — four prompts, ~300 words

1. Of your three diagnosis notes (HW1, HW2, HW3), which is your **strongest**, and why? **Quote one sentence** from it.
2. Where in the loop (observe → locate → hypothesise → inspect → change → verify → explain) did you most often **cut a step**? What did it cost you?
3. Which class of failure was **slowest** for you — paths, environments, notebook state, tracebacks, or data? What is your concrete plan for getting faster next semester?
4. How did your **use of AI change** between HW1 and HW3? One thing you **stopped** doing, one thing you **started**.

300 words is a ceiling. 200 specific words beat 300 general ones.

## The final exam

Separate session, closed book, in reading week — about four weeks from now. A mock exam with a key comes first. New scenarios, same skills.

**The best preparation is re-reading your own three diagnosis notes.**